Revit 管道間自動化檢討外掛 開發計畫書

Oct 18, 2026 · @YiHui

一、專案概述

本專案開發一套 Revit 外掛（C# / Revit API），一鍵完成管道間的法規與空間檢討，並把結果自動寫回模型、上色、出圖與匯出報表，減少人工逐一量測與查核的時間。

開發期為 7 天，最低支援版本為 Revit 2022。本週只做各模組的核心規則與基本自動化，其餘列為未來工作，範圍見第七章。

檢討範圍分為兩個模組：

模組	內容	需要的模型
模組一：淨空間檢討	防火區劃、防火時效、維修門、維修空間（不含配管）	建築模型（牆、門、Room、樓板）
模組二-1：管線分隔與淨距	管線間淨距、電氣與水管分隔、瓦斯管獨立	機電模型（可為連結模型）
模組二-2：管道間避開結構	管道間與管線不與柱、梁衝突，淨空不被結構壓縮	建築＋結構模型（可為連結模型）

預期成果：

- 可在 Revit 2022 以上版本（2022–2026）執行的外掛（含功能區按鈕與檢討視窗）
- 可調整的檢討規則設定檔（JSON）
- 自動化輸出：參數寫回、違規上色、CSV 檢討報表（Excel 可直接開啟）
- 測試模型與檢討報告書

二、法規依據與檢討項目總表

國內法規沒有通用的管道間維修空間尺寸數值，因此每個檢討項目都會標註性質：「法規」代表有明確條文，不合格就是違規；「設計參數」代表依條文精神或事務所標準訂定，門檻值寫在設定檔中，由使用者調整。

編號	檢討項目	依據	性質
A1	垂直貫穿樓板管道間的牆體 ≥ 1 小時防火時效	設計施工編 第79-2條	法規

編號	檢討項目	依據	性質
A2	維修門 ≥ 1 小時防火時效＋遮煙性能	設計施工編 第79-2條	法規
A3	每層有連續樓板且貫穿部位符合第85條時，免適用 A1、A2	內政部111年函釋	法規（判別）
A4	維修孔／維修門不得開向特別安全梯樓梯間	設計施工編 特別安全梯構造條文	法規
A5	地下建築物管道與其他部分 1 小時區劃	設計施工編 地下建築物專章	法規
A6	管道間淨寬、淨深、淨面積	高層專章「應考慮維修及更換空間」	設計參數
A7	維修門淨寬、淨高	同上	設計參數
A8	門前作業淨空、門上方淨高	同上	設計參數
A9	維修門開向共用部分	公寓大廈管理條例 第10條（維修權責）	設計參數
B1	管線之間淨距（含保溫層）	高層專章「維修及更換空間」	設計參數
B2	電氣管道與給排水、消防、燃氣、空調水管分隔	設備編 第2-1條	法規
B3	高層建築物瓦斯管道間單獨設置（與給排水共構除外）	設計施工編 高層專章	法規
B4	風管於管道間開口處設防火閘板	設備編 第94條	法規
C1	管道間範圍與結構柱、梁不衝突	設計協調（施工可行性）	設計參數
C2	管線貫穿梁的位置與尺寸（若允許貫穿）	結構設計規範／結構技師規定	設計參數

條號與現行條文以全國法規資料庫為準；高雄市若有自訂審查要點，另行加入設定檔。

參考：設計施工編 第79-2條 · 設計施工編 全文 · 設備編 全文 · 管道間樓板區劃函釋

三、模組一：淨空間檢討（不含配管）

模組一只讀取建築模型中的 Room、牆、門與樓板，是整個外掛的基礎。模組二會沿用這裡建立的「管道間物件」。

3.1 管道間辨識

1. 收集所有 Room（`OST_Rooms`），依設定檔中的名稱或編號規則篩選（例：`PS`、`DS`、`EPS`、`WS`、`管道間`）。
2. 對每個管道間建立 `ShaftContext`，內容包括：所在樓層、完成面邊界（`SpatialElementBoundaryLocation.Finish`）、邊界牆清單、相連的門、上下樓層的對應管道間。
3. 自動補建：偵測沒有放置 Room 的封閉小空間，提示使用者後自動補建 Room 並命名。

3.2 防火區劃與防火時效（A1–A5）

項目	判斷方式
A1 邊界牆	由 `Room.GetBoundarySegments()` 的 `ElementId` 取得邊界牆，讀取類型參數「防火時效」（`FIRE_RATING` 或自訂共用參數），換算為小時後與 1 hr 比較
A2 維修門	找出 FromRoom 或 ToRoom 為該管道間的門，檢查「防火時效」≥ 1 hr，且自訂共用參數「遮煙性能」= 是
A3 樓板判別	檢查管道間範圍內各層是否有連續樓板（無 Shaft Opening、樓板開口未覆蓋管道間），有樓板者標記為「免適用 A1、A2」並在報表中註明函釋
A4 開向	以 `door.get_FromRoom(phase)` 與 `get_ToRoom(phase)` 取得門另一側的房間，若名稱或「空間類型」參數屬於樓梯間或排煙室，則判定違規
A5 地下層	樓層標高低於設定的地面層時，套用相同的 A1、A2 規則並註明地下建築物條文

3.3 維修門與維修空間（A6–A9）

項目	判斷方式	預設門檻（可調）
A6 淨尺寸	完成面邊界計算最小外接矩形，取短邊為淨寬、長邊為淨深；面積取 `Room.Area`	由使用者設定
A7 門尺寸	讀取門的淨寬、淨高參數；沒有淨寬參數時，以寬度扣除門框寬度推算	由使用者設定

項目	判斷方式	預設門檻（可調）
A8 門前作業淨空	在門外側擠出一個實體盒（寬 = 門寬，深與高由設定），用 `ElementIntersectsSolidFilter` 查詢牆、柱、家具、設備、一般模型是否侵入	深 600–900 mm
A8 門上方淨高	作業盒向上延伸到上層樓板，查詢是否有梁的底部低於設定淨高	由使用者設定
A9 開向共用部分	門另一側房間的「共用/專有」參數或名稱關鍵字（走廊、梯廳、機房）	警告，不判定違規

3.4 垂直連續性

同一編號的管道間逐層比對：把每層的邊界擠出成實體，投影到同一平面後以 `BooleanOperationsUtils` 計算交集面積。重疊率低於設定值（例如 90%）時提出警告，並記錄最大偏移量。

3.5 輸出欄位（寫回共用參數）

`PS_淨寬`、`PS_淨深`、`PS_淨面積`、`PS_檢討結果`（合格／警告／違規）、`PS_違規項目`、`PS_檢討時間`。

四、模組二：機電管線

模組二的機電與結構元件多半在連結模型中，所以第一步是建立「跨模型查詢層」：取得 `RevitLinkInstance` 的連結文件與 `GetTotalTransform()`，把管道間實體轉換到連結模型的座標後再過濾，結果再轉回主模型座標顯示。

一週版本先處理與建築在同一個模型內的機電、結構元件；連結模型的座標轉換列為延後項目。

4.1 模組二-1：管線分隔與淨距

收集管道間內的管線

- 類別：`OST_PipeCurves`、`OST_DuctCurves`、`OST_Conduit`、`OST_CableTray`，以及對應的配件。
- 以管道間實體（邊界擠出至上層樓板）搭配 `ElementIntersectsSolidFilter` 篩選。
- 每支管線記錄：系統分類（給水、排水、消防、瓦斯、空調水、電氣…）、外徑、保溫層厚度、中心線。

項目	判斷方式	預設門檻（可調）
B1 管線淨距	對管道間內的立管兩兩計算中心線距離，扣除兩管的半徑與保溫層厚度後得到淨距	依管徑分級設定
B1 管線與牆淨距	立管中心到管道間完成面邊界的最短距離，扣除半徑與保溫層	由使用者設定
B1 維修側淨空	模組一的門前作業盒向管道間內延伸，檢查是否有管線佔據維修門正後方	由使用者設定
B2 電氣分隔	同一管道間內同時有電氣類（Conduit、Cable Tray）與水類管線時，檢查兩群之間是否有隔牆；沒有則判定違規	法規
B3 瓦斯獨立	高層建築物中，瓦斯管與給排水以外的系統同在一個管道間時判定違規	法規
B4 防火閘板	風管穿出管道間牆的位置，檢查附近是否有防火閘板族群（依族群名稱或參數判斷）	法規

系統分類以 `MEPSystem` 的系統類型名稱比對設定檔中的關鍵字表，避免寫死在程式中。

4.2 模組二-2：管道間避開結構柱梁

項目	判斷方式	結果
C1 管道間與柱	管道間實體與 `OST_StructuralColumns` 求交集，計算侵入體積與侵入後的剩餘淨寬	違規或警告
C1 管道間與梁	管道間實體與 `OST_StructuralFraming` 求交集；梁穿越管道間時，計算梁底到樓板的淨高與被阻擋的平面面積	違規或警告
C2 管線與梁	管線與梁的實體求交集；若結構允許穿梁，檢查穿孔位置與孔徑比例是否符合設定	違規或警告
位置建議	違規時，以管道間邊界與柱梁投影做 2D 差集，提出可移動方向與最小移動距離	建議

位置建議是加分功能：在平面上把柱、梁投影成禁區，向四個方向試算管道間平移，回報第一個不衝突的位置。

五、自動化功能

自動化的目標是讓使用者按一次按鈕，就得到「模型內可見的結果＋可交出去的圖說與報表」，取代人工量測、標註、截圖與整理表格。

一週版本只做前四項與 CSV 報表；自動標註、自動出圖、即時檢核列為未來工作。

功能	做法	節省的人工作業
一鍵全案檢討	掃描所有樓層的管道間，執行勾選的規則	逐一開視圖量測與查核
結果寫回參數	寫入 3.5 節的共用參數，明細表自動更新	手動維護檢討表
違規上色	`OverrideGraphicSettings` 依合格／警告／違規上色；作業盒以 DirectShape 顯示，可一鍵清除	手動畫雲線標示
跳至問題	點選清單中的項目，自動建立剖面框 3D 視圖並選取相關元件	在模型中找位置
自動標註	以牆完成面的 `Reference` 建立淨寬、淨深尺寸，並放置房間標籤	逐一標尺寸
自動出圖	為每個管道間建立放大圖（Callout）、套用視圖樣板、排入圖紙	建視圖、排圖紙
CSV 報表	依樓層與編號輸出 CSV（UTF-8 含 BOM，Excel 開啟中文不亂碼），每列包含檢討值、門檻、結果、法規條號	整理檢討報告附表
即時檢核（進階）	`IUpdater` 監看牆、門、Room 變更，只重算受影響的管道間	修改後重新檢討

所有修改模型的動作包在同一個 `TransactionGroup` 中，使用者可以一次復原整批結果。

六、系統架構與技術選型

架構分為三層：規則引擎不引用 Revit API，可以單獨寫單元測試；Revit 層負責擷取資料與修改模型；UI 層只負責顯示與操作。新增一個檢討項目，只需要新增一個規則類別。

6.1 技術選型

項目	選擇	說明
語言與框架	C#；Revit 2022–2024 用 .NET Framework 4.8，Revit 2025 以上用 .NET 8	一份程式碼、多版本建置（見 6.5）
開發工具	Visual Studio 2022	安裝 .NET Framework 4.8 Developer Pack 與 .NET 8 SDK
UI	一週版本用強制回應 WPF 視窗	避開非強制回應視窗的執行緒問題；ExternalEvent 列為延後
設定檔	JSON（Newtonsoft.Json）	Revit 內建此套件，使用相容版本即可，兩種框架都能用
報表	CSV（UTF-8 含 BOM）	不引入額外套件，避免與 Revit 內建 DLL 版本衝突
規則引擎	`ShaftChecker.Core` 以 netstandard2.0 建置	同時給 net48 與 net8 的 Revit 專案參考
測試	xUnit（規則引擎）＋測試模型	
版本控制	Git＋GitHub	

6.2 專案結構

```
ShaftChecker.sln
├─ ShaftChecker.Core        // 不引用 RevitAPI：規則介面、幾何演算法、結果模型、設定檔讀取
│   ├─ Rules/               // A1_WallFireRating.cs、A8_FrontClearance.cs …
│   ├─ Geometry/            // 最小外接矩形、多邊形交集、淨距計算
│   └─ Models/              // ShaftInfo、DoorInfo、PipeInfo、Issue
├─ ShaftChecker.Revit       // 引用 RevitAPI：資料擷取、連結模型座標轉換、寫回參數、上色、出圖
│   ├─ Collectors/
│   ├─ Writers/
│   └─ Commands/            // IExternalCommand、IExternalApplication（功能區）
├─ ShaftChecker.UI          // WPF 檢討視窗、設定編輯器
├─ ShaftChecker.Tests       // xUnit
└─ config/shaft-rules.json
```

6.3 規則介面

```
public interface IShaftRule
{
    string Id { get; }            // "A1"
    string Name { get; }          // "管道間牆體防火時效"
    RuleNature Nature { get; }    // Regulation 或 DesignParameter
    string Reference { get; }     // "設計施工編 第79-2條"
    IEnumerable<Issue> Check(ShaftInfo shaft, RuleConfig cfg);
}

public record Issue(string RuleId, string ShaftId, Severity Severity,
                    string Message, double? Actual, double? Required,
                    IReadOnlyList<long> ElementIds);
```

`ShaftInfo` 是 Revit 層整理好的純資料（邊界座標、牆的防火時效、門的尺寸、相鄰房間、管線清單），所以規則類別完全不碰 Revit API，可以用假資料測試。

6.4 設定檔範例

```
{
  "shaftNamePatterns": ["^PS", "^DS", "^EPS", "^WS", "管道間"],
  "stairRoomPatterns": ["樓梯", "梯間", "排煙室"],
  "commonRoomPatterns": ["走廊", "梯廳", "機房"],
  "fireRatingParam": "防火時效",
  "smokeSealParam": "遮煙性能",
  "minFireRatingHours": 1.0,
  "minClearWidthMm": 600,
  "minDoorClearWidthMm": 600,
  "frontClearanceDepthMm": 750,
  "frontClearanceHeightMm": 2000,
  "verticalOverlapRatio": 0.9,
  "pipeClearanceMm": [ { "maxDiaMm": 50, "clearMm": 50 }, { "maxDiaMm": 150,
"clearMm": 100 } ],
  "systemGroups": { "Electrical": ["電力", "弱電"], "Gas": ["瓦斯"], "Water":
["給水", "排水", "消防", "冷卻水"] },
  "enabledRules": ["A1", "A2", "A4", "A6", "A8"]
}
```

範例中的數值只是起始值，正式使用前需由組員依參考資料或事務所標準確認，並在報告書中註明來源。

6.5 Revit 2022 以上相容性

用一個 SDK 樣式的專案檔，以建置組態區分 Revit 版本：R22、R23、R24 對應 `net48`，R25、R26 對應 `net8.0-windows`。RevitAPI 參考可用 NuGet 的 `Nice3point.Revit.Api.RevitAPI`、`Nice3point.Revit.Api.RevitAPIUI`（版本號對應 Revit 年份），或手動參考各版本安裝目錄的 DLL。

差異	Revit 2022–2023	Revit 2024 以上	處理方式
元件 Id 數值	`IntegerValue`（int）	`Value`（long），`IntegerValue` 列為過時	擴充方法 `GetIdValue()` 以條件編譯切換
單位	`UnitTypeId`	相同	一律用 `UnitTypeId`，不用舊的 `DisplayUnitType`
執行框架	.NET Framework 4.8	2025 起 .NET 8	只用兩邊都有的 API；`System.Text.Json` 在 net48 需另裝，所以改用 Newtonsoft.Json
.addin 位置	`%AppData%\Autodesk\Revit\Addins\2022`	各年份資料夾	建置後自動複製到對應年份資料夾

```
public static class ElementIdExtensions
{
    public static long GetIdValue(this ElementId id)
    {
#if REVIT2024_OR_GREATER
        return id.Value;
#else
        return id.IntegerValue;
#endif
    }
}
```

在 R24 以上的組態中，於專案檔加入 `<DefineConstants>$(DefineConstants);REVIT2024_OR_GREATER</DefineConstants>`。測試時至少在 Revit 2022 與組內最新版本各執行一次。

七、一週開發時程與分工

開發期只有 7 天，因此範圍收斂為「模組一核心＋模組二簡化版＋基本自動化」，第 3 天結束要有可展示的模組一，第 7 天交付。

範圍	本週必做	延後（報告書列為未來工作）
模組一	管道間辨識、A1、A2、A4、A6、A7、A8	A3 樓板判別、A5 地下層、A9 開向共用部分、垂直連續性
模組二-1	B1 管線淨距、B2 電氣與水管同間	B3 瓦斯獨立、B4 防火閘板、維修側淨空
模組二-2	C1 管道間與柱梁衝突	C2 管線穿梁、位置建議
自動化	一鍵檢討、寫回參數、違規上色、點選跳至元件、CSV 報表	自動標註、自動出圖、即時檢核、非強制回應視窗
模型來源	機電、結構元件與建築在同一模型	連結模型座標轉換

第 3 天交出可展示的模組一，第 7 天完成交付

- **第 1 天** 環境建置（2022–2026 多版本）、共用參數、測試模型、管道間辨識指令
- **第 2 天** A1 牆防火時效、A2 維修門防火與遮煙、A4 不得開向樓梯間
- **第 3 天** A6 淨尺寸、A7 門尺寸、A8 門前作業淨空、結果寫回參數

檢查點：模組一可展示

- **第 4 天** B1 管線淨距（含保溫層）、B2 電氣與水管同間檢查
- **第 5 天** C1 管道間與結構柱梁衝突、測試模型補齊 T8–T10
- **第 6 天** 違規上色、點選跳至元件、CSV 檢討報表
- **第 7 天** 整合測試（2022 與最新版各跑一次）、報告書與展示

開發路線圖 · 4 個階段、2 個檢查點

第 3 天是最重要的檢查點：模組二與自動化都建立在模組一的管道間物件上。第 5 天結束時若模組二還沒完成，直接進入第 6 天的自動化與報表，未完成的規則列為未來工作。

角色	負責內容	主要交付
組長／架構	專案結構、規則介面、Git 合併、整合測試	可執行的外掛主程式

角色	負責內容	主要交付
模組一	管道間辨識、A1–A9、垂直連續性	模組一規則類別與單元測試
模組二	連結模型查詢層、B1–B4、C1–C2	模組二規則類別
自動化與 UI	檢討視窗、上色、CSV 報表、測試模型	UI 與報表、測試模型

組員人數不足四人時，「組長／架構」與「自動化與 UI」可以合併。

八、測試計畫、驗收標準與風險

驗收以「刻意埋設的違規全部被抓到、合格案例沒有誤報」為標準。

8.1 測試模型

建立三層樓的建築模型，另外各做一份機電與結構的連結模型。每層放 2–3 個管道間，並刻意埋設下列案例：

案例	埋設方式	應觸發
T1 合格管道間	1 hr 牆、1 hr 遮煙門、尺寸足夠、門開向走廊	無
T2 牆防火時效不足	其中一面邊界牆改為無防火時效的類型	A1
T3 維修門缺遮煙	門的「遮煙性能」參數設為否	A2
T4 門開向樓梯間	維修門設在樓梯間牆上	A4
T5 淨寬不足	管道間短邊小於門檻	A6
T6 門前有柱	門外側作業區內放置柱或櫃子	A8
T7 上下層錯位	某一層的管道間平移 300 mm	垂直連續性（延後，本週不測）
T8 管線過密	兩支立管淨距小於門檻	B1
T9 電氣與水管同間無隔牆	同一管道間放電纜架與排水管	B2
T10 梁穿越管道間	結構梁跨過管道間範圍	C1

8.2 驗收標準

- T2–T10 全部被正確標示，T1 沒有任何違規。

- 規則引擎的幾何演算法（最小外接矩形、淨距計算、重疊率）有單元測試，全部通過。
- 重新執行檢討時，結果與第一次一致；修改模型後重跑，結果正確更新。
- 三層測試模型的全案檢討在 30 秒內完成，且在 Revit 2022 與組內最新版本各執行一次，結果一致。

8.3 風險與對策

風險	影響	對策
模型中沒有放 Room，或命名不一致	管道間找不到	名稱規則寫在設定檔；提供自動補建 Room 功能
牆、門沒有填防火時效或遮煙參數	A1、A2 無法判斷	判定為「資料不足」而非合格，另列清單提醒補資料
門的 FromRoom／ToRoom 方向錯誤	A4、A8 判斷反向	以門的朝向與兩側房間交叉確認，必要時自動翻轉判斷
連結模型座標轉換錯誤	模組二整個失準	本週不處理連結模型；之後擴充時先寫座標轉換的獨立測試指令
法規門檻沒有明確數值	結果爭議	法規項與設計參數項分開呈現，數值來源寫在報告書
時程不足	模組二做不完	第 3 天先完成模組一可展示版本；第 5 天未完成的規則列為未來工作

九、如何開始

第 1 天的目標只有一個：在 Revit 裡按下按鈕，列出模型中所有管道間及其邊界牆的防火時效。做到這一步，代表環境、資料、辨識邏輯都通了，之後每條規則都是在這個基礎上加功能。

9.1 第 1 天工作清單

- [ ] 以 Revit 2022 為最低版本，另選組內最新版本做相容性測試（2022–2024 用 .NET Framework 4.8，2025 以上用 .NET 8）
- [ ] 安裝 Visual Studio 2022、.NET Framework 4.8 Developer Pack、.NET 8 SDK，以及 RevitLookup（用來查看元件參數，除錯必備）
- [ ] 建立 GitHub 儲存庫，依 6.2 節建立方案與四個專案
- [ ] 建立共用參數檔：`遮煙性能`（是/否）、`PS_淨寬`、`PS_淨深`、`PS_淨面積`、`PS_檢討結果`、`PS_違規項目`
- [ ] 建立測試模型第一層，先放 T1–T6 六個案例，並填好牆類型的「防火時效」
- [ ] 寫出第一個指令 `ListShaftsCommand`（9.3 節），在測試模型上跑通

- [ ] 把 `ListShaftsCommand` 的資料整理成 `ShaftInfo`，寫第一條規則 A1 與它的 xUnit 測試

9.2 建立專案

使用 Nice3point 範本可以省去手動設定 .addin 與除錯路徑：

```
dotnet new install Nice3point.Revit.Templates
dotnet new revit-addin -n ShaftChecker.Revit
```

範本以建置組態區分 Revit 版本，請確認有 R22 組態；若範本版本已不支援 2022，改用下方的手動方式，並依 6.5 節設定多版本建置。

不使用範本時，建立類別庫專案，參考 Revit 安裝目錄下的 `RevitAPI.dll` 與 `RevitAPIUI.dll`（Copy Local 設為否），並在 `%AppData%\Autodesk\Revit\Addins\<版本>\` 放置 .addin 檔：

```
<?xml version="1.0" encoding="utf-8"?>
<RevitAddIns>
  <AddIn Type="Command">
    <Name>管道間清單</Name>
    <Assembly>C:\Dev\ShaftChecker\bin\Debug\ShaftChecker.Revit.dll</Assembly>
    <AddInId>8D83C886-B739-4ACD-A9DB-1BC78F315B2B</AddInId>

<FullClassName>ShaftChecker.Revit.Commands.ListShaftsCommand</FullClassName>
    <VendorId>TEAM01</VendorId>
  </AddIn>
</RevitAddIns>
```

`AddInId` 請用 Visual Studio 的「建立 GUID」重新產生；`VendorId` 改成你們的組別代號。

9.3 第一個可執行的指令

```
using System.Linq;
using System.Text;
using System.Text.RegularExpressions;
using Autodesk.Revit.Attributes;
using Autodesk.Revit.DB;
using Autodesk.Revit.DB.Architecture;
using Autodesk.Revit.UI;

namespace ShaftChecker.Revit.Commands
{
    [Transaction(TransactionMode.ReadOnly)]
    public class ListShaftsCommand : IExternalCommand
    {
        public Result Execute(ExternalCommandData data, ref string message,
```

```
ElementSet elements)
        {
            Document doc = data.Application.ActiveUIDocument.Document;
            var pattern = new Regex("^(PS|DS|EPS|WS)|管道間");   // 之後改讀設定
檔
            var opt = new SpatialElementBoundaryOptions
            {
                SpatialElementBoundaryLocation =
SpatialElementBoundaryLocation.Finish
            };

            var shafts = new FilteredElementCollector(doc)
                .OfCategory(BuiltInCategory.OST_Rooms)
                .WhereElementIsNotElementType()
                .Cast<Room>()
                .Where(r => r.Area > 0)
                .Where(r =>
                {
                    string name =
r.get_Parameter(BuiltInParameter.ROOM_NAME)?.AsString() ?? "";
                    return pattern.IsMatch(name) || pattern.IsMatch(r.Number
?? "");
                })
                .ToList();

            var sb = new StringBuilder();
            foreach (Room room in shafts)
            {
                string name =
room.get_Parameter(BuiltInParameter.ROOM_NAME)?.AsString();
                double areaM2 = UnitUtils.ConvertFromInternalUnits(room.Area,
UnitTypeId.SquareMeters);
                sb.AppendLine($"{room.Level?.Name}  {room.Number} {name}  面
積 {areaM2:F2} m²");

                var wallIds = room.GetBoundarySegments(opt)
                    .SelectMany(loop => loop)
                    .Select(seg => seg.ElementId)
                    .Distinct();

                foreach (ElementId id in wallIds)
                {
                    if (doc.GetElement(id) is Wall wall)
                    {
                        Element wallType = doc.GetElement(wall.GetTypeId());
                        string rating =
```

```
wallType?.get_Parameter(BuiltInParameter.FIRE_RATING)?.AsString();
                        sb.AppendLine($"    牆 {wall.Id}  {wallType?.Name}
防火時效：{(string.IsNullOrEmpty(rating) ? "(未填)" : rating)}");
                    }
                }
            }

            TaskDialog.Show("管道間清單",
                shafts.Count > 0 ? sb.ToString() : "找不到管道間，請檢查 Room 名
稱或編號規則");
            return Result.Succeeded;
        }
    }
}
```

這段程式只讀不寫，所以用 `TransactionMode.ReadOnly`。如果邊界牆在連結模型中，`seg.ElementId` 會是連結實例，需要改讀 `seg.LinkElementId`，連結模型列為延後項目，本週請把結構與機電元件放在同一個測試模型中。

這段程式只用到 Revit 2022 起就有的 API（`UnitTypeId`、`SpatialElementBoundaryOptions`），可直接在 2022–2026 建置執行。

9.4 第 2 天起

1. 把指令中的資料擷取搬到 `ShaftChecker.Revit/Collectors`，產出 `ShaftInfo`。
2. 在 `ShaftChecker.Core/Rules` 實作 A1，並用假的 `ShaftInfo` 寫 xUnit 測試。
3. 依序加入 A2、A4（第 2 天）與 A6、A7、A8（第 3 天），每加一條規則就在測試模型上驗證對應案例。
4. 規則都能跑之後，再做結果寫回參數與上色，完成第 3 天的可展示版本。